Embedded Systems Architectures - Laboratory Guide 4

I2C Sensors (and actuators)

Academic Year 2025/2026

Authors: Arnaldo Oliveira, Fábio Coutinho

1. Objectives

- Configuration and use of an I2C bus
- Reading and writing to a simple temperature sensor (TC74) via the I2C bus
- Interaction with a multi-sensor (DHT20) via the I2C bus

2. Introduction

The use of efficient and low-power communications is crucial to support the expansion of the Internet of Things (IoT). In this sense, the choice of a communication protocol to connect microcontrollers, sensors, and actuators is a determining factor for the success of any embedded project. Among the various options, the I2C (Inter-Integrated Circuit) bus is a fundamental technology due to its simplicity and economy.

Using only two communication lines – the serial data line (SDA) and the clock line (SCL) – drastically reduces wiring and printed circuit board (PCB) complexity. In a world of increasingly miniaturized and component-dense IoT devices, this minimalist characteristic translates into reduced costs and space, which are essential for the viability of devices such as smart sensors, wearables, and remote actuators. More than just saving pins, the I2C bus allows multiple devices (from temperature and humidity sensors to memory modules and analog-to-digital converters) to share a single pair of wires, differentiating themselves by unique addresses. The multi-master and multi-slave nature of I2C promotes a modular, scalable architecture,

allowing developers to quickly integrate new sensors or features without completely redesigning the system.

In this class, we will explore the use of the I2C protocol to read temperature and humidity data from sensors and to change their operating settings.

3. Practical Work

The practical work in this class is based on the Reference API for the ESP32-C6 peripherals and the ESP32-C6 Technical Reference Manual: https://documentation.espressif.com/esp32-c6_technical_reference_manual_en.pdf

Additionally, the **technical data sheets for the TC74 and DHT20 sensors**, available on the course website at elearning.ua.pt, **should be consulted**.

3.1 I2C-TOOL - Exploring I2C communications with simple sensors

In this section, we will use the *i2c-tool* provided in the ESP-IDF framework examples to demonstrate how to develop I2C-related applications. This example provides a command-line tool to configure the I2C bus, search for devices, read and write registers, and examine them.

Start by assembling the circuit in Figure 1 on the whiteboard. As you can see, the goal is to develop a circuit using the TC74 temperature sensor, including two 4.7 kΩ pull-up resistors.

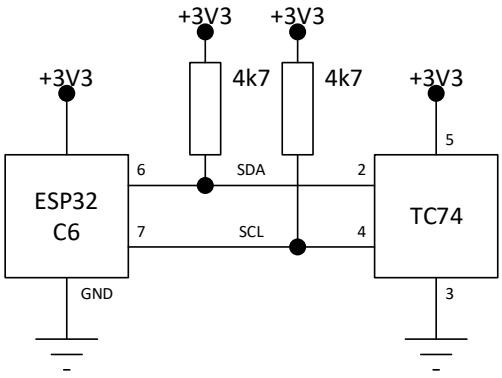

Figure 1 - I2C connection circuit to the TC74 sensor.

Warning! Always be cautious and check the connections on the breadboard before powering the ESP32.

Next, using the ESP-IDF extension wizard, create a new project based on the *i2c -> i2c_tools* example. Then, compile the example and download it to the board.

If needed, before launching the Monitor, change the USB port you are using to ensure you have a dedicated serial interface for the command-line tool. Once you power the board via the USB-UART port, activate the Monitor and confirm that you see the following information:

```
I (186) main_task: Calling app_main()
================================================================
| Steps to Use i2c-tools |
| |
| 1. Try 'help', check all supported commands |
| 2. Try 'i2cconfig' to configure your I2C bus |
| 3. Try 'i2cdetect' to scan devices on the bus |
| 4. Try 'i2cget' to get the content of specific register |
| 5. Try 'i2cset' to set the value of specific register |
| 6. Try 'i2cdump' to dump all the register (Experiment) |
| |
================================================================
Type 'help' to get the list of commands.
Use UP/DOWN arrows to navigate through command history.
Press TAB when typing command name to auto-complete.
I (366) main_task: Returned from app_main()
i2c-tools>
```

The **`i2c-tools>`** command prompt indicates that the i2c-tools application running on the board is ready to receive one of the 6 supported commands: **`help`**, **`i2cconfig`**, **`i2cdetect`**, **`i2cget`**, **`i2cset`**, and **`i2cdump`**.

Question 1: Study each of the supported commands and check which functionality is associated with it. Then, run the **`i2cdetect`** command and check the result. You should get a table similar to the one shown below. How do you interpret the result and why does this occur?

```
i2c-tools> i2cdetect
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
```

The next step is to configure, using the **`i2cconfig`** command, which ports are connected to the clock (SCL signal) and serial data (SDA signal), which can be done as follows:

```
i2c-tools> i2cconfig --sda 6 --scl 7
```

Run the **`i2cdetect`** command again and you should see the following result (or similar, depending on the particular variant of the TC74 you are using):

```
i2c-tools> i2cdetect
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- 48 -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
```

Question 2: What can you conclude from this result? In which address is the temperature sensor accessible?

Question 3: Analyzing the sensor's technical data sheet and the operation of the *i2c-tool*, execute the command that allows you to read the instantaneous ambient temperature. The result of the command should contain the temperature expressed in hexadecimal as illustrated below.

```
i2c-tools> <command to use>
0x19 <--- Example of result
```

Question 4: Heat the temperature sensor by pressing both sides with your fingers for a while. Repeat the temperature reading command and you should observe slight changes in the reading.

Question 5: Analyze the operating mode (standby/normal) of the TC74 sensor by reading its configuration register. Interpret the result, which you should observe to be similar to the following:

```
i2c-tools> <command to use>
0x40 <--- Example of result
```

Question 6: Set the sensor to standby mode. You should observe the following result:

```
i2c-tools> <command to use>
I (157146) cmd_i2ctools: Write OK
```

Question 7: Reheat the temperature sensor by pressing both sides again with your fingers for a while. Repeat the temperature reading command. What do you observe?

The process described so far allows you to interact with a simple sensor, reading values from registers and, when supported, writing values to the sensor registers.

3.2 I2C-TOOL: I2C communications with multiple sensors

Warning! Whenever you want to make electrical changes to the breadboard, disconnect it from the power supply.

Assemble the circuit shown in Figure 2 on the breadboard. As documented, the goal is to add the DHT20 temperature sensor to the I2C bus, keeping the two existing 4.7 kΩ pull-up resistors.

Turn the board back on and run the **`i2cdetect`** command, and you should see a result similar to the following. As you can see, two I2C devices are detected on the bus: the TC74 (0x48) and the DHT20 (0x38) – the TC74 address depends on the particular variant you are using.

```
i2c-tools> i2cdetect
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- 38 -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- 48 -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
i2c-tools>
```

Troubleshooting! If you reset the board, you will need to reconfigure the SDA and SCL pins using the **`i2cconfig`** command.

Question 8: Review the DHT sensor's datasheet. Can you explain how to perform a temperature and humidity reading from this sensor? What will be the sequence of bytes to be sent to the sensor and how to read the result?

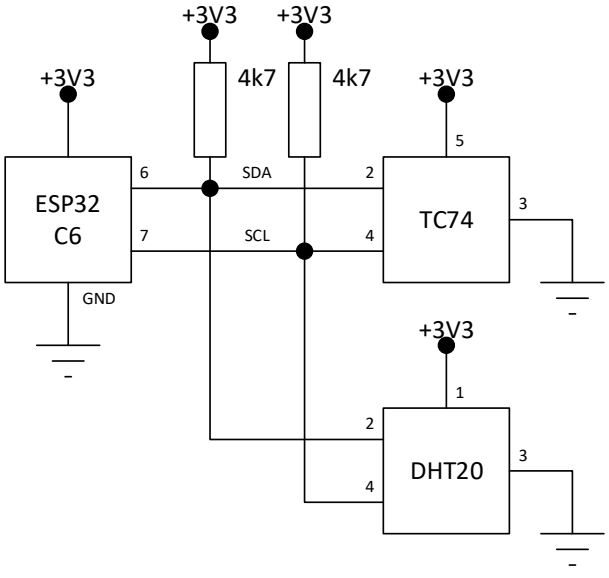

Figure 2 - I2C connection circuit to TC74 and DHT20 sensors.

Question 9: Using the **`i2cset`** and **`i2cget`** commands, take a reading of the temperature and humidity. Convert the result to obtain the respective temperature and humidity.

3.3 Accessing an I2C sensor from a custom software application

This section aims to develop a program that retrieves the temperature from the TC74 sensor every 2 seconds and prints the result to the Monitor, in degrees Celsius. Next, part of the code from the file "tc74_test.c," available on the course website, is shown.

```
#include ....

/* ---- I2C configuration ---- */
#define I2C_MASTER_SCL_IO          <TBD>
#define I2C_MASTER_SDA_IO          <TBD>
#define I2C_MASTER_NUM             I2C_NUM_0
#define I2C_MASTER_FREQ_HZ         <TBD>
#define I2C_MASTER_TX_BUF_DISABLE  0
#define I2C_MASTER_RX_BUF_DISABLE  0
#define I2C_MASTER_TIMEOUT_MS      1000

/* ---- TC74 specifics ---- */
#define TC74_ADDR                  <TBD>
#define TC74_REG_TEMP              <TBD>
#define TC74_REG_CONFIG            <TBD>
...
void app_main(void)
{
   uint8_t tempReg, cfgReg;
   i2c_master_bus_handle_t busHandle;
   i2c_master_dev_handle_t devHandle;

   i2c_master_init(&busHandle, &devHandle);
   ESP_LOGI(TAG, "I2C initialized successfully");

   // Read configuration register
   ESP_ERROR_CHECK(TC74RegisterRead(devHandle, TC74_REG_CONFIG,
                                    &cfgReg, 1));
```

```
    ESP_LOGI(TAG, "Initial config = 0x%02X", cfgReg);

    // Ensure sensor is in normal mode (bit7 = 0)
    if (cfgReg & 0x80)
    {
        ESP_LOGW(TAG, "Sensor in standby, switching to normal mode");
        cfgReg &= 0x7F;
        ESP_ERROR_CHECK(TC74RegisterWriteByte(devHandle,
                                              TC74_REG_CONFIG,
                                              cfgReg));
        vTaskDelay(pdMS_TO_TICKS(200));
    }

    while (1)
    {
        ESP_ERROR_CHECK(TC74RegisterRead(devHandle, TC74_REG_TEMP,
                                         &tempReg, 1));
        int8_t tempSigned = (int8_t)tempReg; // Temperature in C
        ESP_LOGI(TAG, "Temperature: %d C", tempSigned);
        vTaskDelay(pdMS_TO_TICKS(2000));
    }

    // Normally we never reach this, but here for completeness:
    ESP_ERROR_CHECK(i2c_master_bus_rm_device(devHandle));
    ESP_ERROR_CHECK(i2c_del_master_bus(busHandle));
    ESP_LOGI(TAG, "I2C de-initialized successfully");
}
```

Question 10: Study the I2C reference API, particularly the resource allocation and release component. Check how a record is read.

Question 11: Study and complete the code provided in the file "*tc74_test.c*" (modifying the **TBD** values) to make it compatible with the TC74 sensor. Test the code and heat the sensor with hot air, confirming that the result is similar to the following:

```
I (243) TC74: I2C initialized successfully
I (253) TC74: Initial config = 0x40
I (253) TC74: Temperature: 25 C
I (2253) TC74: Temperature: 25 C
```

```
I (4253) TC74: Temperature: 25 C
I (6253) TC74: Temperature: 25 C
I (8253) TC74: Temperature: 26 C
I (10253) TC74: Temperature: 26 C
```

Question 12: Starting from the provided code, make the necessary changes to make it compatible with taking readings from the DHT20 sensor. Test the code and heat the sensor, confirming that the result is similar to the following:

```
I (8453) DHT20: Temperature: 22.60 C, Humidity: 69.61 %
I (10493) DHT20: Temperature: 22.61 C, Humidity: 69.64 %
I (12533) DHT20: Temperature: 22.76 C, Humidity: 71.17 %
I (14573) DHT20: Temperature: 23.49 C, Humidity: 74.63 %
I (16613) DHT20: Temperature: 23.94 C, Humidity: 77.58 %
I (18653) DHT20: Temperature: 24.24 C, Humidity: 80.10 %
I (20693) DHT20: Temperature: 24.37 C, Humidity: 82.08 %
```